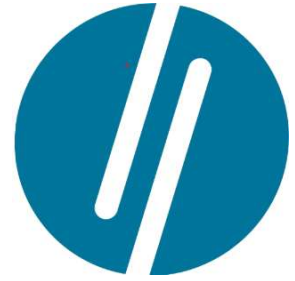


Universidad Nacional de General Sarmiento
Instituto de industria
Licenciatura en Sistemas



TRABAJO PRÁCTICO

Elizabeth's Quest



Materia: Programación 1, comisión 03

Profesores: Lucas Bidart Gauna
Leonardo Dávalos

Alumnos:

Navas Joana	DNI:42.591.059	navasjoa@gmail.com
Maidana Ulpo Mara Carolina	DNI: 41.172.499	maidanaumc@gmail.com
Patroni Tomas	DNI: 43.993.262	patronitomas06@gmail.com

Año 2026

Introducción

El presente trabajo práctico tuvo como objetivo aplicar los conceptos fundamentales de Programación Orientada a Objetos mediante el desarrollo de un videojuego en lenguaje Java utilizando la biblioteca Entorno. A lo largo del proyecto se pusieron en práctica conocimientos relacionados con la creación de clases, encapsulamiento, interacción entre objetos, manejo de eventos, estructuras de datos y lógica de videojuegos en tiempo real. El desarrollo permitió integrar los contenidos abordados durante la cursada en una aplicación completa y funcional.

El videojuego desarrollado consiste en una aventura de plataformas protagonizada por una princesa que debe atravesar un escenario compuesto por múltiples islas flotantes generadas de manera aleatoria. Durante su recorrido, la protagonista debe evitar o eliminar distintos tipos de enemigos, superar obstáculos y desplazarse a través de diferentes niveles del mapa hasta alcanzar un castillo ubicado al final del escenario. Para enriquecer la experiencia de juego se incorporaron mecánicas como doble salto, proyectiles, enemigos con distintos comportamientos, plataformas especiales y un sistema de vidas.



Figura 1 Pantalla de inicio del juego.



Figura 2 Mapa de juego.

Durante el desarrollo se enfrentaron diversos desafíos técnicos relacionados con la generación procedural del mapa, la implementación de colisiones, el seguimiento de cámara, la inteligencia básica de los enemigos y la integración de recursos gráficos personalizados. La resolución de estos problemas permitió obtener un producto final más completo y robusto, además de consolidar los conocimientos adquiridos en la materia mediante la aplicación práctica de técnicas de programación y diseño orientado a objetos.

Descripción

A continuación, se detallan las clases implementadas, seguidas de los problemas encontrados y las decisiones de diseño tomadas durante el desarrollo.

Comenzamos con explorar los archivos dados e investigando un poco por fuera, respetando las reglas que nos dieron, una vez conforme con la investigación comenzamos.

Clases implementadas

El proyecto se estructura en paquetes lógicos para mantener la organización y responsabilidades de cada objeto bien delimitadas.

Clases principales, núcleo del juego y problemas surgidos (con solución):

- ***Juego.java***: Es la clase principal del proyecto, en la misma implementamos diversos atributos propios, como los arreglos que contienen las islas, los distintos tipos de enemigos, u otros que contienen objetos que se muestran en momentos particulares del juego como el castillo y los regalos. En esta clase también se creó `private int cámaraX` para tener seguimiento de la princesa, el cual guarda el desplazamiento horizontal de la cámara. Implementamos esto porque el mapa es mas grande que la pantalla la forma que fue pensada es como que el personaje principal se movía y si no había nada que lo siguiera, entonces el personaje se perdería y dejaría de ser visible para el jugador, entonces lo que hicimos fue crear la variable `camaraX` que esto también nos ayudó que la princesa permaneciera en el centro de la toma y de esta forma el jugador podía ver lo que tenía por delante.
- ***Islas.java (posteriormente Bloques.java)*** : Esta clase almacena un objeto de tipo `Rectangle` en la variable `área` y variables de tipo boolean que distinguen las características especiales de cada bloque.



Figura 3. Tipos de islas. De arriba a abajo se observan las islas de bloques destruibles (bloques de grava), las islas estándar (bloques de césped) e islas inestables (bloques de arena).

Además, para que cada partida sea diferente implementamos la Clase Random para definir la cantidad de islas por nivel, la cantidad de bloques por isla (conjunto de bloques) y la distancia entre las mismas.

En un principio las islas se generaban y almacenaban en un único arreglo, atributo de la clase Juego, pero presentaba algunas fallas como por ejemplo se generaban islas a distintos niveles pero nunca en la misma coordenada x, sino separadas como un camino único. Además al implementar el personaje y los movimientos, a veces atravesaba una isla y otras no ya que no detectaba correctamente las colisiones cuando la princesa caía en estas. Como solución a estos problemas implementamos una matriz, lo que permitió una mejor organización para la generación del mapa y, a su vez, para la detección de colisiones.

Más adelante, dicha clase, pasa a llamarse Bloques, para asignarle un nombre más acertado con lo implementado realmente, grupos de bloques que forman islas.

- **Princesa.java**: Esta clase es la encargada de la gran mayoría de funciones y atributos de la princesa, así como saltar, disparar con un Click, desplazarse horizontalmente, la asignación de vida con el que inicia el juego y luego la condición de pérdida de vidas hasta llegar a 0.

Uno de los problemas, en relación con esta clase y la anterior, fue que al crear la matriz de islas la princesa no llegaba al siguiente nivel al saltar. Por esta razón, se implementó el doble salto, el cual se logró al implementar el atributo totalSalto, el cual cuenta la cantidad de saltos realizados, limitándose a un máximo de 2 saltos seguidos. De esta forma se evita que la princesa pueda saltar infinitamente. Los mismos los recupera al colisionar con el piso de una isla.

- **AereoNormal.java** y **AereoRapido.java**: Los enemigo aéreo normal y rápido fueron diseñados como las amenazas básicas del juego. Su comportamiento consiste en desplazarse horizontalmente a una velocidad constante a través de diferentes alturas predeterminadas del escenario. Estos enemigos aparecen de manera periódica durante la partida y pueden provenir tanto desde el lado izquierdo como desde el derecho de la pantalla. Su implementación permitió introducir un desafío dinámico para el jugador, obligándolo a esquivar obstáculos en movimiento mientras avanza por el mapa.



Figura 4. Tipos de Enemigos, de izquierda a derecha se observan los enemigos rápidos seguidos de los enemigos normales..

un problema que se presentó a la hora de la implementación de las entidades aéreas fue el conflicto entre la generación aleatoria de los niveles de las islas y la generación de los enemigos ya que estos aparecían en todas las posiciones posibles dentro del eje y (entre 0 y 600) una solución simple y a la vez prolija fue implementar “alturas válidas” donde estos enemigos pudieran aparecer sin chocar con las islas, se añadieron las alturas válidas una vez obtenidos los niveles donde se dibujarán las islas.

Otro problema que surgió durante el desarrollo de los enemigos aéreos estaba relacionado con su generación y dibujo. Inicialmente, los enemigos se generaban utilizando las coordenadas de la ventana, lo que provocó comportamientos incorrectos una vez implementado el sistema de la cámara, ya que podían aparecer fuera del área visible para el jugador. Además, fue necesario adaptar el mecanismo que detectaba cuándo un enemigo abandonaba la pantalla para que tuviera en cuenta el desplazamiento de la cámara. Finalmente, al incorporar sprites animados, se debió implementar un sistema que seleccionara distintas imágenes según la dirección de movimiento, permitiendo representar correctamente el desplazamiento hacia la izquierda o hacia la derecha.

- **EnemigoTerrestre.java:** El enemigo terrestre fue desarrollado para patrullar las plataformas del nivel inferior del escenario. A diferencia de los enemigos aéreos, estos permanecen apoyados sobre las islas y se desplazan entre los límites de cada una, invirtiendo automáticamente su dirección al alcanzar un borde. Para su generación fue necesario implementar un sistema capaz de identificar cada isla completa y asociar un único enemigo a ella. Este comportamiento aporta variedad al juego, ya que obliga al jugador a prestar atención tanto a las amenazas provenientes del aire como a los obstáculos presentes en las plataformas por las que debe desplazarse.

Uno de los problemas encontrados durante el desarrollo fue la generación de los enemigos terrestres. Inicialmente, los enemigos se creaban tomando como referencia cada bloque individual del nivel, lo que provocaba que aparecieran varios enemigos sobre una misma isla. El objetivo era generar únicamente un enemigo por isla completa, independientemente de la cantidad de bloques que la componían. Para resolver este inconveniente, se implementó un algoritmo capaz de detectar el inicio y el final de cada isla analizando la continuidad de los bloques. De esta manera, se identificó cada isla como una única estructura y se generó un solo enemigo terrestre asociado a sus límites, permitiendo además que el enemigo haga patrullas correctamente entre los extremos de la plataforma.

- **Proyectil.java:** La clase Proyectil fue desarrollada para permitir que la protagonista pueda defenderse de los enemigos presentes en el escenario. Cada proyectil se genera en la posición actual de la princesa cuando el jugador realiza un clic con el mouse y se dirige hacia la ubicación señalada por el cursor. Para lograr este comportamiento se calculó un vector de dirección a partir de la posición de origen y destino, permitiendo que el proyectil se desplace de manera fluida en cualquier dirección del plano. Además, se implementaron detecciones de colisión con enemigos y bloques del escenario, provocando la eliminación del proyectil al impactar contra alguno de ellos. Esta mecánica incorporó un componente de precisión y estrategia al juego, brindando al jugador una herramienta para enfrentar las distintas amenazas que aparecen durante la partida.

Durante el desarrollo de la clase Proyectil surgieron algunos inconvenientes relacionados con la dirección de los disparos. Como el proyectil debía dirigirse hacia la posición del mouse, fue necesario calcular correctamente su trayectoria para que se desplazara en cualquier dirección. Además, al implementar la cámara del juego, se presentaron errores debido a la diferencia entre las coordenadas del escenario y las coordenadas visibles en pantalla. Una vez corregido este problema, también se añadieron las colisiones con enemigos y bloques para que el proyectil desapareciera al impactar contra ellos.

- **Regalo.java:** La clase Regalo fue implementada para incorporar un elemento de recompensa dentro del juego. Su función es generar un objeto coleccionable con forma de corazón que aparece en determinados momentos de la partida y puede ser recogido por la princesa. Para ello, la clase almacena información sobre su posición, el momento en que debe aparecer y si se encuentra activo o no dentro del escenario. Además, se implementó la lógica necesaria para dibujar el objeto utilizando una imagen personalizada y para detectar cuándo deja de estar visible en pantalla. Esta mecánica permite ofrecer beneficios al jugador y aporta mayor variedad a la experiencia de juego.



Figura 5. Objeto bonificador en forma de corazón el cual otorga una vida extra a la princesa cuando interactúa con él (desaparece después de un tiempo si no se lo agarra).

Castillo y condición de victoria

El castillo representa el objetivo final del juego y se encuentra ubicado al final del mapa. La misión principal del jugador consiste en atravesar los distintos obstáculos y enemigos hasta llegar a esta estructura. Para determinar la victoria se implementó una detección de colisión entre la princesa y el castillo, de modo que al alcanzarlo se activa una pantalla especial de finalización de partida. Además de funcionar como meta del recorrido, el castillo contribuye a reforzar la ambientación y la narrativa del juego, ya que simboliza la conclusión exitosa de la aventura.



Figura 5. Castillo ubicado al final del mapa el cual activa la condición de victoria al interactuar con el

Durante la implementación de la condición de victoria surgieron inconvenientes relacionados con la detección de colisiones. En algunas ocasiones la princesa parecía llegar visualmente al castillo, pero la victoria no se activaba debido a diferencias entre la posición mostrada en pantalla y la utilizada por la lógica del juego. Para resolver este problema fue necesario ajustar correctamente la zona de detección y verificar su funcionamiento junto con el sistema de cámara.

un problema relativamente importante se originó a la hora de la implementar el método de java Intersects, ya que no se tuvo en cuenta que el método dibujarrectángulo tomaba en cuenta las posiciones X e Y desde el centro e intersects tomaba en cuenta desde la esquina superior izquierda esto generaba un conflicto a nivel código y desencadenaba diversos bugs a la hora de ejecutar el juego. esto se resolvió recalculando la lógica de las colisiones ahora si teniendolas en cuenta a partir de la esquina superior izquierda, la forma en la que se dibujaban se mantenía igual

Uno de los desafíos más importantes del proyecto fue la implementación del sistema de colisiones entre la princesa y las plataformas. Debido a que el mapa se generaba de manera aleatoria y estaba compuesto por múltiples niveles de islas, fue necesario desarrollar una lógica capaz de diferenciar colisiones laterales, superiores e inferiores. Durante las primeras pruebas se detectaron problemas como atravesar plataformas, detecciones incorrectas al golpear bloques desde abajo y errores en el reinicio de los saltos al aterrizar. La solución consistió en calcular por separado los desplazamientos horizontales y verticales, verificando las intersecciones con los bloques antes de actualizar la posición definitiva del personaje.

Generación del escenario

Con el objetivo de evitar que las partidas resulten repetitivas, se implementó un sistema de generación aleatoria del escenario utilizando la clase Random de Java. Este sistema determina la cantidad de islas presentes en cada nivel, la cantidad de bloques que componen cada una y la distancia existente entre ellas. Gracias a esta estrategia, cada nueva partida presenta una distribución diferente del mapa, aumentando la rejugabilidad del juego.

Además, se incorporaron distintos tipos de bloques. Los bloques normales constituyen la mayor parte del escenario, mientras que los bloques rompibles pueden ser destruidos por la princesa al golpearlos desde abajo y los bloques inestables desaparecen luego de ser pisados, generando nuevos desafíos para el jugador.

Sistema de colisiones

La implementación de las colisiones representó uno de los aspectos más complejos del proyecto. Para ello se utilizaron las variables velocidadX y velocidadY de la clase Princesa, permitiendo calcular la nueva posición del personaje antes de aplicarla definitivamente. De esta manera fue posible detectar colisiones laterales, superiores e inferiores con las plataformas del escenario.

Las colisiones laterales impiden que la princesa atraviese los bloques y provocan una caída al detener su movimiento horizontal. Por otro lado, las colisiones verticales permiten que el personaje se apoye correctamente sobre las plataformas, recupere la capacidad de saltar y pueda interactuar con bloques especiales. Para el resto de las interacciones del juego, como las colisiones entre enemigos, proyectiles, regalos y el castillo, se utilizó el método intersects de la clase Rectangle proporcionada por Java.

Para verificar la colisión con las islas, se calculan las coordenadas de los límites del personaje y de las islas asumiendo una forma geométrica rectangular. Luego, se recorre el arreglo de niveles que almacena las respectivas alturas de las islas. El sistema comprueba si la princesa se encuentra en la altura correspondiente a uno de los niveles (para colisiones en el eje X) o si se halla completamente en el espacio vertical entre islas (para colisiones en el eje Y). En ese punto, se recorre el arreglo de bloques del nivel en cuestión.

Si el personaje colisionó con un bloque, el comportamiento varía según el borde impactado: Borde inferior (bloque por encima del personaje) se verifica si el bloque es rompible; en caso afirmativo, se le asigna el valor null. Borde superior (suelo) Se verifica si el bloque es inestable si lo es, se le asigna un tick de caída, se activa la bandera caídaActiva y se almacena la dirección del desplazamiento de la princesa al momento del impacto. Además, al colisionar con el suelo, el personaje recupera la capacidad de saltar.

Por otro lado, la verificación de colisiones en el eje Y se divide en dos métodos según el sentido del movimiento: si el personaje cae, se evalúa la colisión con el borde superior de las

islas; si sube, se comprueba el borde inferior. Ambos métodos comparten una lógica inicial que recorre las distintas alturas de las islas; si el personaje se encuentra completamente en el espacio libre entre dos islas, se pasa a evaluar el siguiente nivel. De lo contrario, se recorren los bloques del nivel correspondiente y se verifica si los laterales del personaje están contenidos entre los límites laterales de los bloques.

Ante una colisión, ambos métodos finalizan seteando la variable velocidadY en cero y ajustando la nueva coordenada Y a la altura correspondiente, rozando el borde superior o inferior según el caso. Finalmente, estos métodos retornan dicha coordenada Y actualizada para corregir la posición del personaje; de lo contrario, se mantiene el desplazamiento original.

En consonancia con los bloques inestables, se desarrolló un método en la clase Juego llamado actualizarBloquesInestables. Este método recorre la matriz de bloques y verifica si algún elemento con la bandera caídaActiva alcanzó su tick de caída. Para lograr un efecto dominó, se identifica el bloque subsiguiente según la dirección de la caída. Para asegurar que este pertenezca al mismo grupo de la isla, se comprueba que la distancia entre la coordenada X de ambos bloques no supere los 40 píxeles. De esta forma, se activa la caída del próximo bloque antes de asignar null al bloque actual.

Para el resto de las colisiones (Princesa-enemigos, proyectil-enemigos, proyectil-islas y princesa-castillo), se utilizó el método intersects de la clase Rectangle de Java, dado que estos casos no requerían de un control lógico particular o personalizado.

Conclusiones finales

A partir de lo expuesto en el presente informe, se puede concluir que los objetivos planteados para el trabajo práctico fueron alcanzados satisfactoriamente. A pesar de las distintas dificultades que surgieron durante el desarrollo, se logró implementar todas las funcionalidades principales requeridas, obteniendo un videojuego que no solo es completo y funcional sino que también presenta al usuario un desafío dinámico interactivo y divertido.

Por último, el trabajo en equipo resultó fundamental para alcanzar los objetivos propuestos. La colaboración entre los integrantes permitió distribuir tareas, intercambiar ideas y resolver dificultades de manera conjunta, favoreciendo tanto el aprendizaje individual como la obtención de un resultado final de mayor calidad.

Lecciones aprendidas:

- **Trabajo en equipo:** Durante el desarrollo de este proyecto aprendimos no solo a colaborar entre nosotros como individuos sino también a coordinarnos a la hora de planificar tareas, distribuir responsabilidades y cumplir con los plazos establecidos. Además de aprender de las distintas formas de pensar de cada integrante del grupo, las cuales nos permitieron entregar el proyecto de manera ordenada y en un formato limpio.
- **Programación Orientada a Objetos:** Aprendimos a dividir el proyecto en múltiples clases con responsabilidades específicas, facilitando la organización, reutilización y mantenimiento del código.
- **Sistema de colisiones:** Comprendimos la importancia de detectar correctamente las interacciones entre los distintos elementos del juego para lograr movimientos y comportamientos realistas.
- **Manejo del movimiento:** Implementamos desplazamientos utilizando velocidades independientes en los ejes horizontal y vertical, lo que permitió desarrollar mecánicas como el salto, la caída y el movimiento lateral.
- **Generación aleatoria de escenarios:** Utilizamos la clase Random para crear mapas diferentes en cada partida, aumentando la variedad y la rejugabilidad del juego.
- **Uso de arreglos y matrices:** Aprendimos a almacenar y administrar grandes cantidades de objetos, como bloques, enemigos e islas, mediante estructuras de datos eficientes.
- **Implementación de una cámara dinámica:** Desarrollamos un sistema de cámara que sigue al personaje principal, comprendiendo la diferencia entre las coordenadas del mundo y las coordenadas visibles en pantalla.
- **Diseño de enemigos con distintos comportamientos:** Implementamos enemigos aéreos y terrestres con mecánicas propias, lo que nos permitió comprender mejor la programación de comportamientos dentro de un videojuego.
- **Integración de recursos gráficos:** Reemplazamos figuras geométricas por sprites, fondos e imágenes personalizadas, mejorando significativamente la presentación visual del proyecto.
- **Depuración y resolución de errores:** A lo largo del desarrollo fue necesario identificar, analizar y corregir numerosos errores, fortaleciendo nuestra capacidad para resolver problemas de programación.
- **Trabajo colaborativo con Git y GitHub:** Aprendimos a utilizar herramientas de control de versiones para compartir código, integrar cambios realizados por distintos integrantes y mantener un historial del proyecto.
- **Planificación y escalabilidad del código:** Comprendimos la importancia de diseñar una estructura organizada desde el inicio para facilitar futuras mejoras y la incorporación de nuevas funcionalidades.